

Git Workflow

Version: 1.0

Purpose

This document defines the Git workflow for Trading Platform Pro.

The goal is to keep the repository clean, reviewable and maintainable throughout the project's lifetime.

Workflow Principles

Every commit should be:

- atomic
- reviewable
- traceable
- reversible

Never mix unrelated changes.

Standard Workflow

```

Analyze

↓

Implement

↓

Run Tests

↓

Run Ruff

↓

git add

↓

git commit

↓

git push

```

Daily Development Workflow

For every task:

1. Understand the requirement.
2. Analyze the affected files.
3. Implement the smallest meaningful change.
4. Execute unit tests.
5. Verify formatting and linting.
6. Review the changes.
7. Commit.
8. Push.

Branch Strategy

Default branch:

```

main

```

Feature branches may be used for larger developments:

feature/

Examples:

feature/order-engine

feature/plugin-system

feature/runtime-monitor

Commit Messages

Commit messages should be concise and descriptive.

Examples:

Improve configuration loader

Add runtime health checks

Refactor message dispatcher

Implement scheduler abstraction

Add unit tests for cache service

Avoid:

fix

update

changes

misc

Recommended Commit Size

One commit should represent one logical change.

Good examples:

- implement one feature
- fix one bug
- refactor one component
- improve one document

Before Commit

Verify:

- Tests passed
- Ruff passed
- Documentation updated (if required)
- No debug code
- No commented-out code
- No temporary files

Push Policy

Push only after:

- successful tests
- successful linting
- review of modified files

Pull Requests

Every Pull Request should:

- solve one problem
- have a clear title
- include an explanation
- remain reasonably small

Repository Hygiene

Do not commit:

- IDE settings (unless project-wide)
- temporary files
- log files
- cache files
- generated artifacts
- secrets
- credentials

Recovery

If a mistake is committed:

1. Identify the problem.
2. Create a dedicated corrective commit.
3. Avoid rewriting shared history unless absolutely necessary.

Branch Naming Convention

Use the following branch prefixes:

- feature/
- fix/
- refactor/
- docs/
- test/
- release/
- hotfix/

Examples:

- feature/order-engine
- docs/developer-guide
- fix/runtime-monitor

Merge Policy

- Never commit directly to `main`.
- Work in feature or documentation branches.
- Merge only after review.
- Keep commits atomic.

Documentation Commits

Documentation changes should be grouped by topic.

Examples:

- docs: improve README
- docs: extend developer guide

- docs: update architecture documentation

Avoid mixing documentation and code changes in the same commit whenever possible.

Versioning

Use Semantic Versioning:

MAJOR.MINOR.PATCH

Examples:

- 1.0.0
- 1.1.0
- 1.1.1

Update CHANGELOG when creating a release.

Related Documents

- AGENTS.md
- Development_Guidelines.md
- Coding_Standards.md
- Testing_Strategy.md